Informe Técnico Plataforma Eventify

Caso semestral de Base de Datos No Relacionales

Integrantes: Dairys Bernal, Amaru Burdiles, Florencia Soto

1. Contexto y Objetivo

La empresa Eventify Global requiere modernizar su plataforma tecnológica para soportar el crecimiento en la venta de entradas para eventos masivos (conciertos, festivales, obras de teatro, etc.). Su infraestructura relacional previa presentaba caídas de rendimiento en los picos de demanda: salidas a venta de artistas masivos y campañas tipo Cyber Day, donde miles de usuarios consultan y compran simultáneamente sobre los mismos registros.

El problema no es el volumen de datos, sino el patrón de acceso. Al poseer un modelo relacional, se exigía reconstruir la ficha de un evento bajo JOIN entre recinto, tipo de entrada y stock; bajo carga, estos JOIN y sus bloqueos se volvían el cuello de botella. Como solución ante esta ineficiencia, se optó por escalar verticalmente.

La migración a MongoDB busca que la unidad de lectura coincida con la unidad de negocio, permitiendo que la ficha de un evento sea un solo documento recuperable en una sola lectura.

El presente informe documenta el modelo implementado en solucion.mongodb.js, justifica las decisiones y detalla la batería de consultas analíticas.

2. Modelo de Datos

La base de datos no-sql-ev2 se compone de cuatro colecciones, todas creadas con reglas de validación $jsonSchema en db.createCollection()

colección	Rol	Patrón Aplicado
usuarios	Clientes y administradores	Documento plano
eventos	Catálogo, recinto y configuración de entradas	Embebido (ubicación, tickets)
ordenes_compra	Registro de transacciones	Referencia (usuario_id, evento_id)
review_evaluaciones	Calificaciones y comentarios post-evento	Referencia (usuario_id, evento_id)

2.1. Usuarios

Almacena la identidad del cliente. Sus campos obligatorios son: run, dv, nombre, apellido, correo, teléfono, fecha_nacimiento y el dato booleano es_vip.

La validación es deliberadamente estricta, ya que es el punto de entrada a datos externos:

- run: Numérico, sin puntos ni guion. De 7 a 8 dígitos y que no comience en cero.
- dv: Un único carácter, aceptando el dígito verificador “K”.
- Correo: Patrón de validación de formato de correo electrónico.
- fecha_nacimiento: de tipo date, permitiendo cálculos de edad y segmentación etaria.
- es_vip: Permite segmentación comercial.

Se decidió separar run por run y dv como dos campos independientes en lugar de un string único, permitiendo validar cada uno con su propia regla y evitando ambigüedades de formato.

2.2. Eventos

Es la colección central y la que concentra decisiones de diseño. Sus campos obligatorios son: código, título, tipo, capacidad, fecha_hora_ini, fecha_hora_fin y activo.

Incorpora dos estructuras embebidas:

- Ubicación (Objeto embebido): Requiere país, ciudad, calle, n_calle y coordenadas, más un detalle opcional. Las coordenadas se validan como un arreglo de exactamente dos elementos numéricos, en el orden de longitud y latitud.
- Tickets (Arreglo de Objetos Embebidos): Cada elemento requiere tipo, precio, stock_ini y disponibles, con zona opcional (platea, palco, general, etc.). La distinción entre stock_ini y disponibles es intencional, ya que permite conservar ambos valores y permite el cálculo de ventas y porcentaje de ocupación por diferencia, sin necesidad de recorrer el histórico de órdenes.

2.3. Ordenes_compra

Registra las transacciones. Sus campos obligatorios son: usuario_id, evento_id, total y fecha.

Usuario_id y evento_id son objectId (referenciados) y operan como claves foráneas hacia usuarios y eventos. El campo total admite double o int con mínimo de 0, y fecha es de tipo date para permitir agrupaciones y filtros temporales.

2.4. Review_evaluaciones

Calificaciones y comentarios de asistentes a eventos finalizados. Requiere de usuario_id, evento_id y calificación, el documento comentario es opcional

La calificación se valida como int entre 0 y 5.

3. Decisiones de Modelado

3.1. Embebidos y motivos

Se embebió ubicación y tickets dentro de eventos bajo los siguientes motivos:

- Siempre se leen juntos: Por concepto, un evento no existe sin lugar donde ocurrir ni sin entradas que vender. Cuando el usuario abre la ficha de un evento, necesita el título, la dirección y los precios en la misma pantalla. Embeberlos, en lugar de referenciarlos, convierte esa pantalla en una sola lectura de un solo documento. En un modelo relacional, este JOIN podía saturar el sistema en el pico de demanda, al ser la operación más repetida en el sistema.
- Cardinalidad acotada: Un evento tiene exactamente una ubicación y un número reducido y predecible de tipos de entrada (general, platea, palco, VIP). El arreglo no crece de forma indefinida.
- Baja volatilidad: La dirección de un recinto y la estructura de precios se definen al publicar el evento y prácticamente no cambian.
- Consistencia en la escritura: Al vivir en un mismo documento, descontar stock y leer el precio ocurren sobre una única unidad atómica. No existe riesgo de que una entrada quede asociada a un evento inexistente.

Si dos eventos distintos ocurren en el mismo recinto, la ubicación se duplica en ambos documentos, siendo una desnormalización deliberada: la dirección de un recinto es un dato inmutable, de modo que el riesgo de incontinencia por duplicación es mínimo. La alternativa (una colección de recintos) solo se justifica si el negocio necesitara administrar recintos como entidad propia con su propia agenda de disponibilidad, etc.

3.2. Referenciados y motivos

Los datos referenciados se aplican bajo ObjectID, siendo estos vínculos ordenes_compra y review_evaluaciones hacia usuarios y eventos.

- Crecimiento ilimitado: Un evento masivo puede acumular decenas de miles de órdenes y reseñas. Embeberlas dentro del documento del evento lo haría crecer sin techo y degradaría cada lectura de la ficha al arrastrar datos que nadie está consultando en ese momento.
- Volatilidad alta y escritura concurrente: Las órdenes se insertan de forma masiva y continua durante la venta. Mantenerlas como documentos independientes distribuye la escritura en lugar de concentrar la contención en un único documento de evento.
- Relaciones de muchos a muchos: Un usuario compra en muchos eventos y un evento es comprado por muchos usuarios. Ninguna de las dos entidades contiene naturalmente a la otra.
- Consulta independiente: Las órdenes y las reseñas se consultan por sí mismas en reportes de ventas, en análisis de satisfacción, etc., sin necesidad de cargar la entidad principal. La referencia permite esos análisis sin arrastrar el documento completo del evento.
- Un dato y una fuente: El correo de un usuario existe en un solo lugar. Si cambia, se actualiza una vez y queda reflejado en todas sus órdenes históricas, sin recorrer registros duplicados.

Reconstruir el detalle de una compra exige un $lookup (equivalente a JOIN), con su costo asociado. Es un costo que se paga en consultas analíticas, no en la navegación del catálogo.

3.3. Regla de diseño resultante

Se embebe lo que se lee junto, es acotado y cambia poco. Se referencia lo que crece sin límite, se escribe constantemente y se consulta por sí mismo.

3.4. Impacto en escalabilidad

El modelo favorece la escalabilidad horizontal. Las colecciones de alto crecimiento (orden_compra, review_evaluaciones) contienen documentos pequeños y uniformes, aptos para la distribución eficiente. La colección eventos se mantiene de tamaño estable y acotado, permitiendo replicarla ampliamente y servir el catálogo desde cualquier nodo con una sola lectura.

4. Batería de Consultas

4.1. Consulta 1 — Detalle de compras por usuario

Objetivo: Obtener el historial de compras de un usuario identificado por RUN, limitado a eventos de una ciudad determinada, con su gasto total.

Pipeline: Parte del $match por run y dv, que reduce el conjunto a un documento antes de cualquier unión: filtrar primero y unir después es lo que mantiene barata la consulta. Un primer $lookup trae sus órdenes y $unwind las convierte en documentos individuales; un segundo $lookup incorpora el evento de cada orden, con su respectivo $unwind. Luego se filtra por ubicacion.ciudad, aprovechando que la ubicación está embebida y no requiere una tercera unión. Finalmente, $group consolida el gasto con $sum, arma el nombre completo con $concat y construye el desglose con $push, mientras $project descarta los campos auxiliares.

4.2. Consulta 2 — Reporte de ventas por evento

Objetivo: Entregar, por evento, el total recaudado, los tickets vendidos, el porcentaje de ocupación y una etiqueta de rendimiento.

Pipeline: Se construye en tres $addFields encadenados que calculan progresivamente sobre el arreglo embebido:

1. $sum sobre tickets.stock_ini y tickets.disponibles obtiene el stock total y el remanente. Un $map recorre el arreglo y calcula, por cada tipo de entrada, el subtotal (stock_ini − disponibles) × precio.
2. $sum consolida los subtotales en el recaudado total, y una resta entrega los tickets vendidos.
3. Un $cond calcula el porcentaje de ocupación protegiendo la división: si stock_total no es mayor que cero, devuelve 0 en lugar de fallar.

El $project final redondea el porcentaje con $round y asigna la etiqueta mediante un $cond: “Exitoso” si la ocupación alcanza el 60%, “Por Mejorar” en caso contrario.

La consulta no utiliza ningún $lookup. Todo el reporte se resuelve dentro del propio documento del evento, porque la decisión de conservar stock_ini junto a disponibles permite deducir las ventas sin recorrer las órdenes.

4.3. Consulta 3 — Eventos futuros con entradas bajo un monto

Objetivo: Localizar eventos aún no realizados en una ciudad, que conserven algún tipo de entrada disponible por debajo de un precio determinado.

Pipeline: El $match combina tres condiciones: $expr con $$NOW compara fecha_hora_ini contra la fecha del sistema para descartar eventos pasados; el filtro por ubicación.ciudad opera sobre el objeto embebido; y $elemMatch exige que un mismo elemento del arreglo tickets cumpla simultáneamente con tener disponibles >= 1 y precio menor al monto.

El uso de $elemMatch es la decisión técnica relevante: sin él, MongoDB daría por válido un evento donde un ticket agotado cuesta poco y otro disponible es caro, porque ambas condiciones se cumplirían en elementos distintos.

El $project renombra campos a términos de negocio (tipo_evento, fecha_inicio, fecha_finalizacion), aplana la ubicación y aplica $filter para devolver únicamente los tipos de entrada que satisfacen el criterio, no el arreglo completo.

4.4. Consulta 4 — Relación precio-satisfacción por evento

Objetivo: Construir un informe que, para cada evento, presente el precio promedio de sus entradas y la calificación promedio obtenida, exponiendo así lo que se cobra y cómo el usuario lo percibe.

A las tres consultas obligatorias, que miden el desempeño comercial, se incorpora la percepción del asistente, aprovechando el registro de review_evaluaciones. El hallazgo que busca exponer es el evento de precio alto con calificación baja, el cual puede ser invisible para cualquier reporte de ventas, y esto puede afectar si el evento se repite y el prejuicio de mala experiencia perdura y las ventas decaen.

Pipeline: El $lookup une eventos con review_evaluaciones por id y eventoid, depositando las coincidencias en el arreglo reviews. Esta unión es necesaria ya que las reseñas se modelaron como colección referenciada y no embebida.

El $project resuelve ambas métricas y aprovecha la característica propia de MongoDb que permite la devolución del arreglo de todos los valores pedidos sobre el cual $avg puede operar directamente. $avg sobre tickets.precio calcula el precio promedio sin recorrer el arreglo de forma explícita, lo que es viable únicamente porque los tickets están embebidos en el propio documento; de haberse referenciado, el mismo cálculo habría exigido un segundo $lookup. La misma mecánica aplica $avg sobre reviews.calificacion, y ambos resultados se redondean con $round a dos decimales.

$cond protege el cálculo de la calificación, verificando. Mediante $size, que reviews contenga al menos un elemento y evitando de aquella forma la división por cero que $avg produciría sobre un arreglo vacío y sustituyendo el

resultado por la etiqueta “Sin evaluaciones”. Devolver null o cero sería, técnicamente, válido pero semánticamente incorrecto, ya que un cero podría confundirse con una calificación deficiente cuando en realidad representa la ausencia de datos.

El arreglo completo de reseñas no se proyecta y solo se obtiene el promedio, ya que los comentarios individuales no aportan al análisis comparativo y solo incrementan el volumen del resultado. La salida conserva código, título y tipo como identificadores de negocio más las dos métricas calculadas.